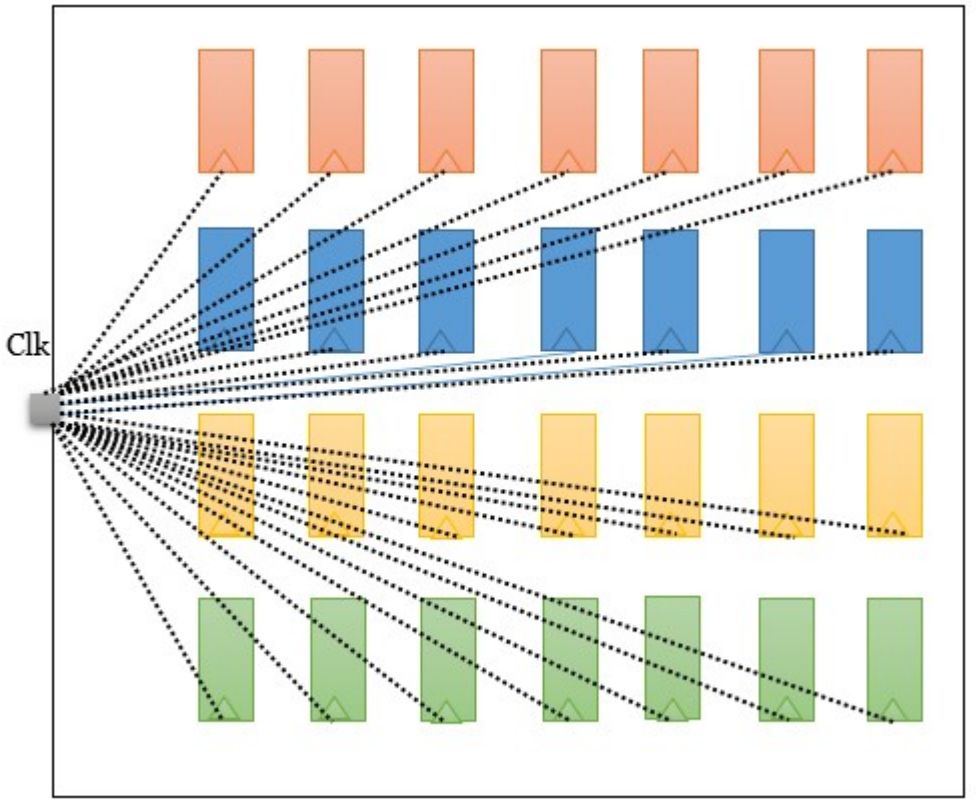
CLOCK TREE SYNTHESIS (CTS)

- Clock is not propagated before CTS so after clock tree build in CTS stage we consider hold timings and try to meet all hold violations
- After placement we have position of all standard cells and macros and in placement we have ideal clock (for simplicity we assume that we are dealing with a single clock for the whole design)
- At the placement optimization stage buffer insertion and gate sizing and any other optimization techniques are used only for data paths but in the clock path nothing we change.

CLOCK TREE SYNTHESIS (CTS)

- CTS is the process of connecting the clocks to all clock pin of sequential circuits by using inverters/buffers in order to balance the skew and to minimize the insertion delay.
- All the clock pins are driven by a single clock source. Clock balancing is important for meeting all the design constraints.

CLOCK TREE SYNTHESIS (CTS) (in this figure clock tree is not built)



Checklist before CTS:

- Before going to CTS it should meet the following requirements:
- The clock source are identified with the create_clock or create_generated_clock commands.
- The placement of standard cells and optimization is done.
- {NOTE: use check_legality –verbose command to verify that the placement is legalized. If cells are not legalize the qor is not good and it might have long run time during CTS stage}
- Power ground nets- pre-routed
- Congestion- acceptable
- Timing – acceptable
- Estimated max tran/cap – no violations
- High fan-out nets such as scan enable, reset are synthesized with buffers.

Inputs required for CTS:

- Placement def
- Target latency and skew if specify (SDC)
- Buffer or inverters for building the clock tree
- The source of clock and all the sinks where the clock is going to feed (all sink pins).
- Clock tree DRC (max Tran, max cap, max fan-out, max no. of buffer levels)
- NDR (Nondefault routing) rules (because clock nets are more prone to cross-talk effect)
- Routing metal layers used for clocks.

Output of CTS:

- CTS def
- Latency and skew report
- Clock structure report
- Timing Qor report

CTS target:

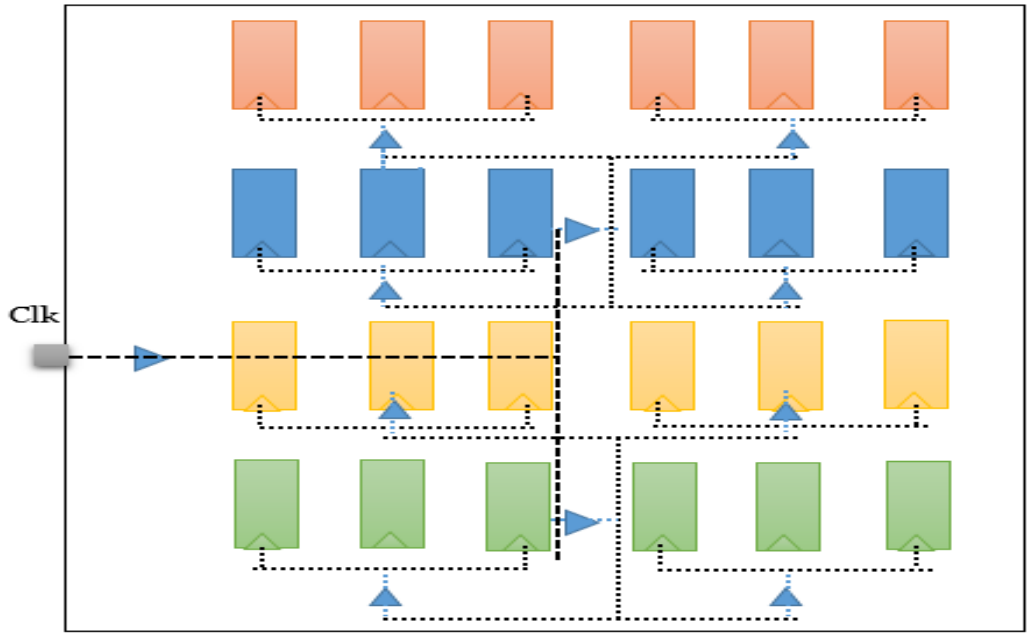
- Skew
- Insertion delay

CTS goal:

- Max Tran
- Max cap
- Max fan-out
- A buffer tree is built to balance the loads and minimize skew, there are levels of buffer in the clock tree between the clock source and clock sinks.

Effect of CTS:

- Clock buffers are added congestion may increase non-clock cells may have been moved to less ideal locations can introduce timing and tran/cap violations.



Checks after CTS:

- In latency report check is skew is minimum? And insertion delay is balanced or not.
- In qor report check is timing (especially HOLD) met, if not why?
- In utilization report check Standard cell utilization is acceptable or not?
- Check global route congestion?
- Check placement legality of cells.
- Check whether the timing violations are related to the constrained paths or not like not defining false paths, asynchronous paths, half-cycle paths, multi-cycle paths in the design.

- **Clock Endpoints types:**
- When deriving the clock tree, the tool identifies two types of clock endpoints:
- **Sink pins (balancing pins):**Sink pins are the clock endpoints that are used for delay balancing. The tool assign an insertion delay of zero to all sink pins and uses this delay during the delay balancing.
- During CTS, the tool uses sink pins in calculations and optimizations for both design rule constraints for both design rule constraints and clock tree timing (skew & insertion delay).
- **Sink pins are:**
- A clock pin on a sequential cell
- A clock pin on a macro cell

Ignore pins:

- These are also clock endpoints that are excluded from clock tree timing calculations and optimizations. The tool uses ignore pins only in calculation and optimizations for design rule constraints.
- During CTS the tool isolate ignore pins from the clock tree by inserting a guide buffer before the pin. Beyond the ignore pins the tool never performs skew or insertion delay optimization but it does perform design rule fixing

- **Ignore pins are:**
- Source pins of clock trees in the fanout of another clock
- Non clock inputs pins of sequential cells
- Output ports
- **Float pins**: it is like stop pins but delay on the clock pin, macro internal delay.
- **Exclude pins:** CTS ignores the targets and only fix the clock tree DRC (CTS goals).
- **Nonstop pin**: by this pin clock tree tracing the continuous against the default behavior. Clock which are traversed through divider clock sequential elements clock pins are considered as non-stop pins.

Why clock routes are given more priority than signal nets:

- Clock is propagated after placement because the exact location of cells and modules are needed for the clock propagation for the estimation of accurate delay, skew and insertion delay. Clock is propagated before routing of signals nets and clock is the only signal nets switches frequently which act as sources for dynamic power dissipation.

CTS Optimization process:

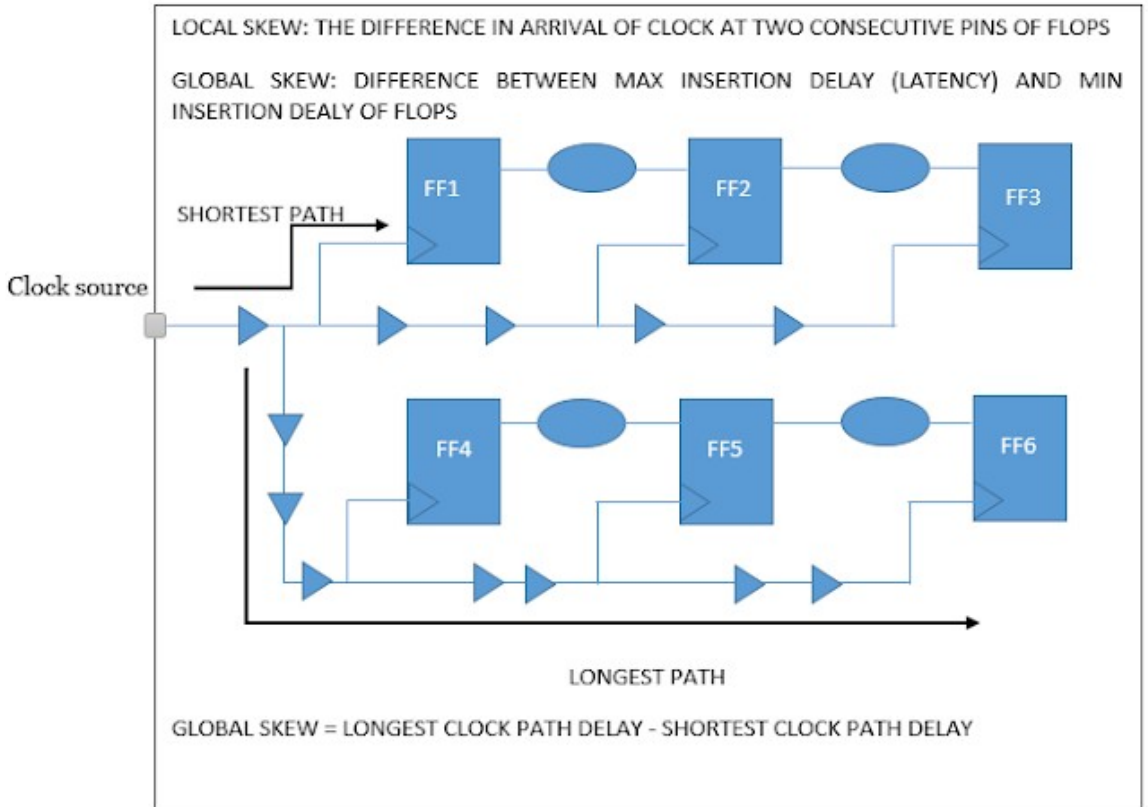
- By buffer sizing
- Gate sizing
- Buffer relocation
- Level adjustment
- HFN synthesis
- Delay insertion
- Fix max transition
- Fix max capacitance
- Reduce disturbances to other cells as much as possible.
- Perform logical and placement optimization to all fix possible timing.

NOTE

- mainly try to improve setup slack in preplacement, inplacement and postplacement optimization before cts stages and in these stages neglecting the hold slack
- in post placement optimization after cts stages the hold slack is improved. as a result of cts lot of buffers are added.

Skew:

- This phenomenon in synchronous circuits. The Difference in arrival of clock at two consecutive pins of a sequential element.

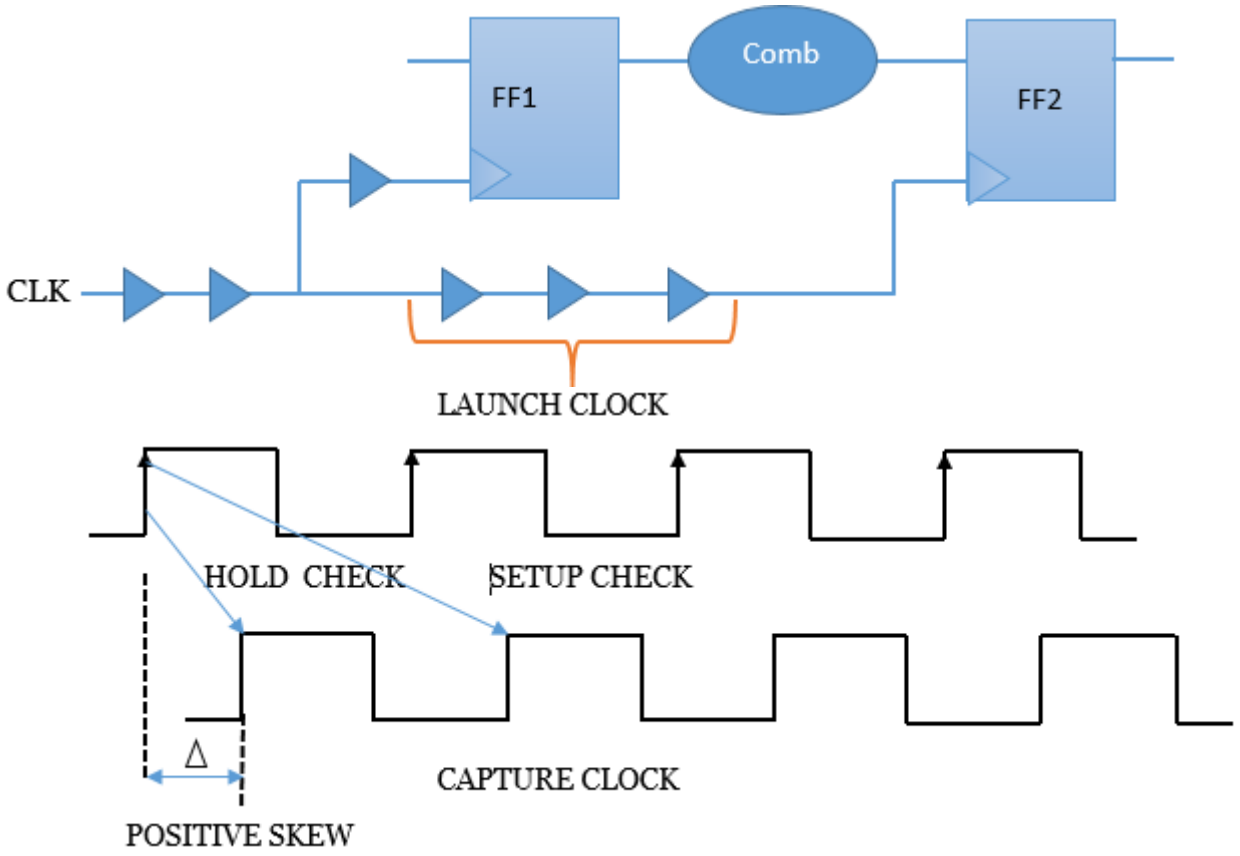


Sources of skew:

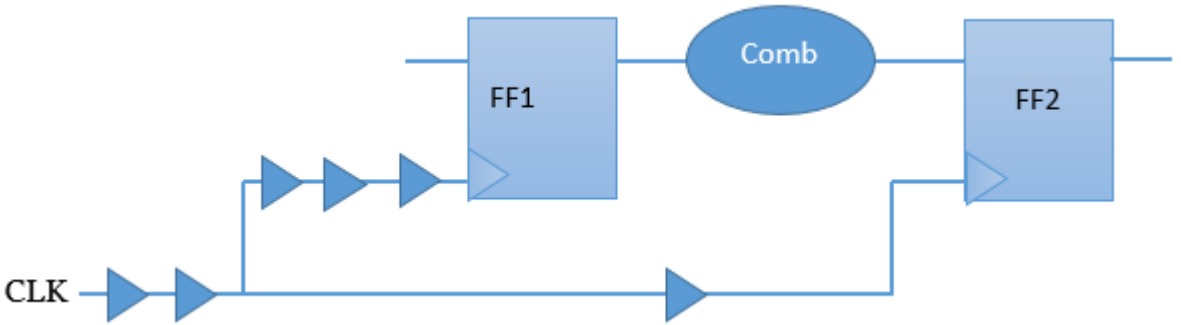
- Wire interconnect length
- Capacitive loading mismatch
- Material imperfections
- Temperature variations
- Differences in input capacitance on the clock inputs

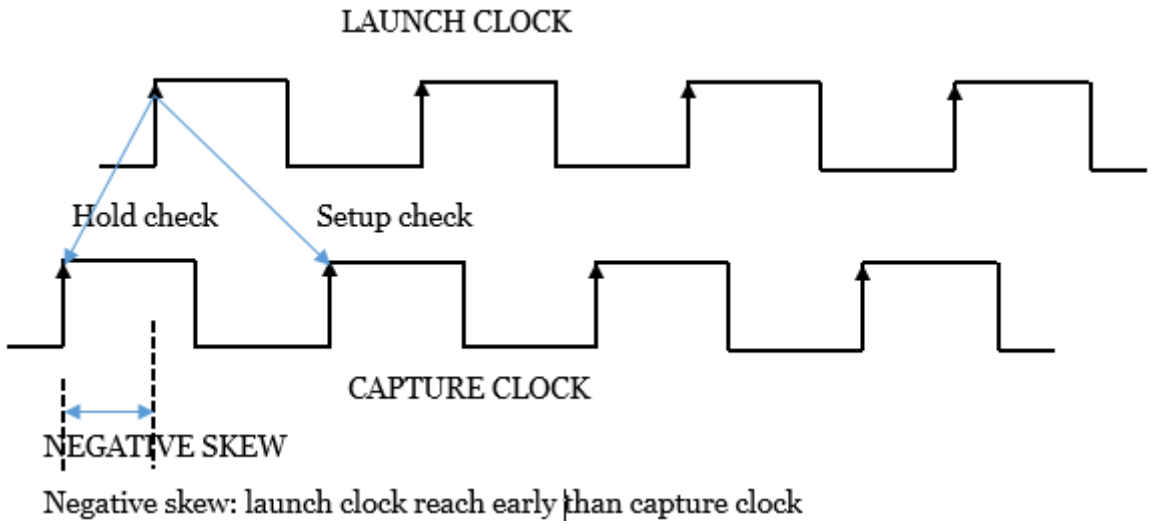
Types of clock skew:

- **Positive skew:** if the capture clock comes late than the launch clock.



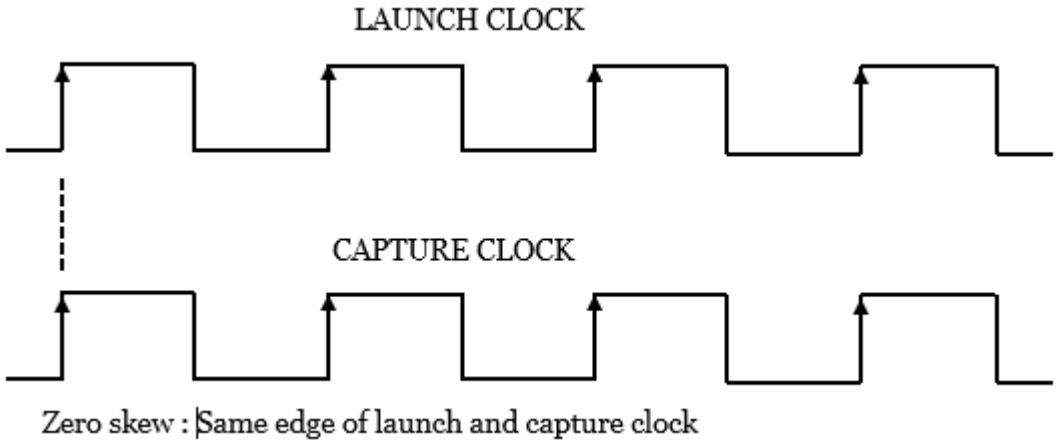
- **Negative skew:** if the capture clock comes early than the launch clock.

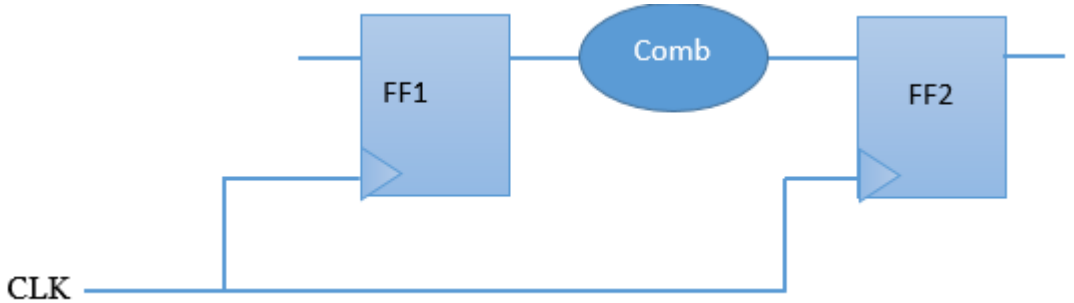




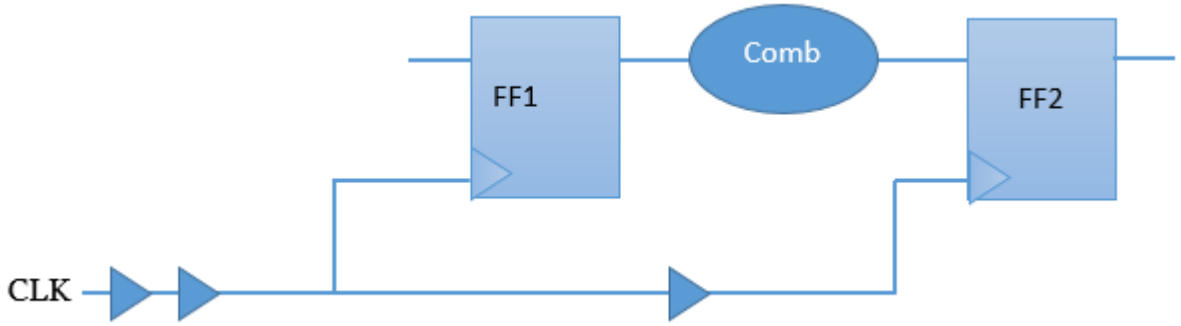
Negative skew: launch clock reach early than capture clock

- **Zero skew:** when the capture clock and launch clock arrives at the same time. (ideally, it is not possible)



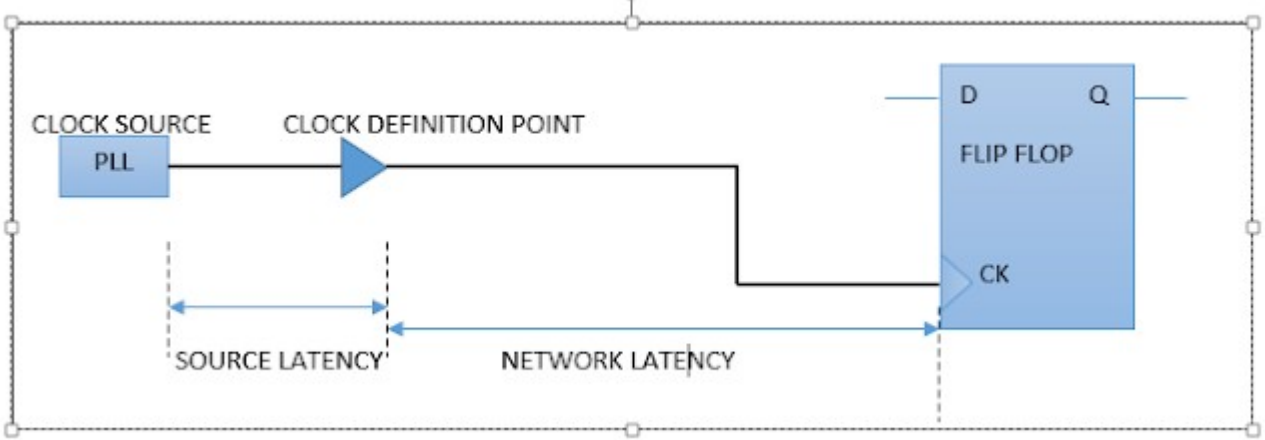


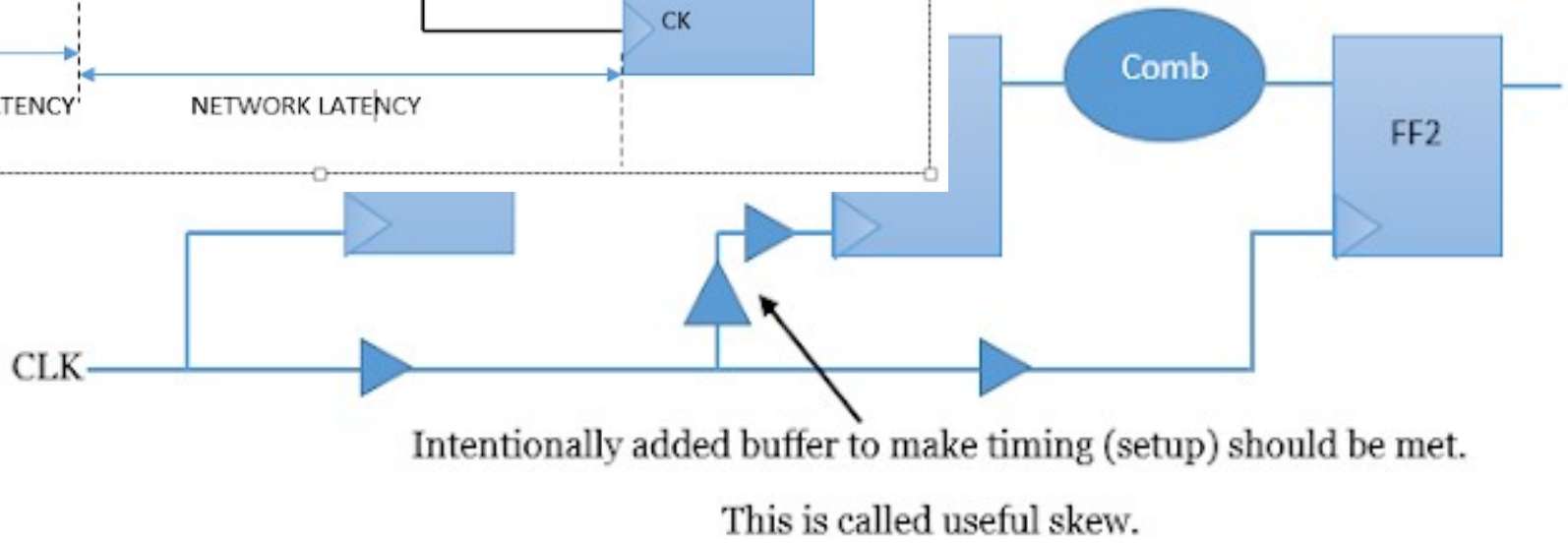
- **Local skew:** difference in arrival of clock at two consecutive pins of sequential element.it can be positive and negative local skew also.



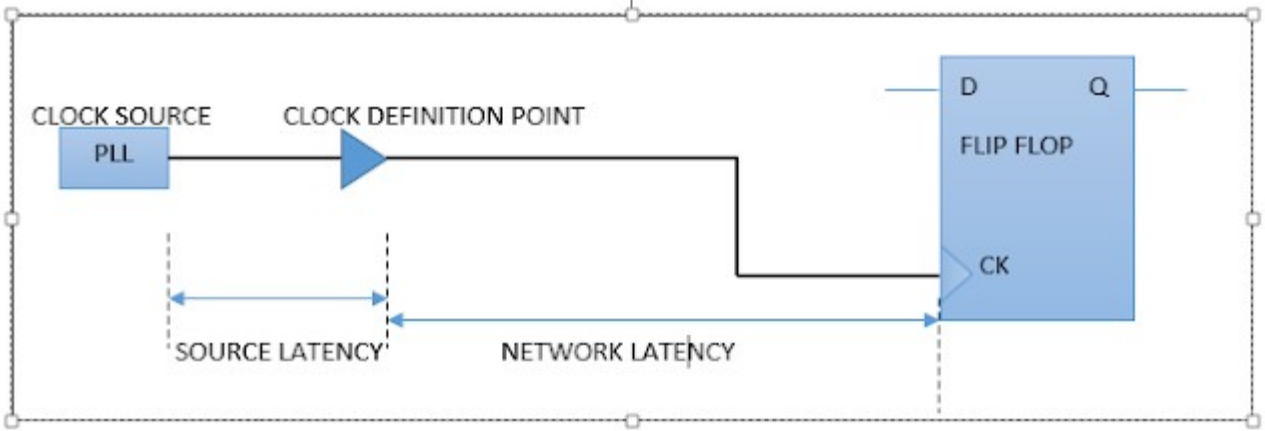
- **Global skew:** the difference between max insertion delay and the min insertion delay.it can be positive and negative local skew also
- **max insertion delay:** delay of the clock signal takes to propagate to the farthest leaf cell in the design.
- **min insertion delay:** delay of the clock signal takes to propagate to the nearest leaf cell in the design.

ed intentionally to resolve setup





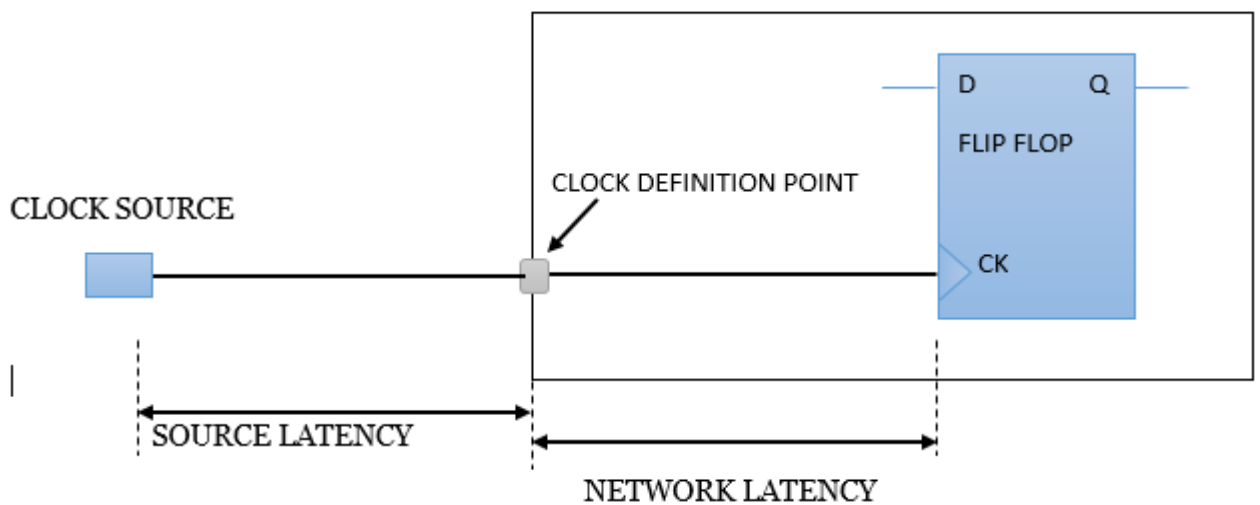
- **Latency: The** delay difference from the clock generation point to the clock endpoints



There are two types of latency:

Source latency: Source latency is also called insertion delay. The delay from the clock source to the clock definition points. Source latency could represent either on-chip or off-chip latency.

Network latency: The delay from the clock definition points(create_clock) to the flip-flop clock pins **.**

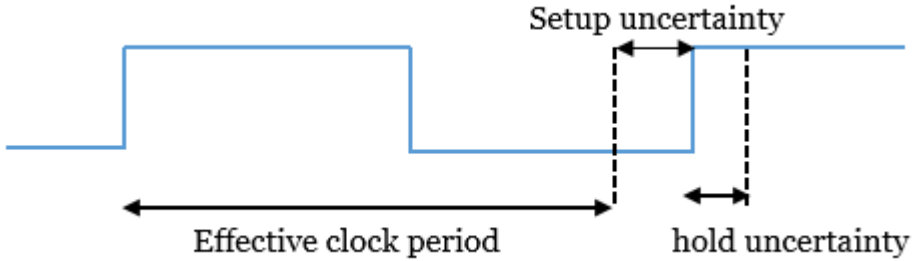


- Set_clock_latency 0.8 [get_clocks clk_name1] ----> network latency
- Set_clock_latency 1.9 –source [get_clocks clk_name1] -------> source latency
- Set_clock_latency 0.851 –source –min [get_clocks clk_name2] -----> min source latency
- Set_clock_latency 1.322 –source –max [get_clocks clk_name2] ------> max source latency
- One important distinction to observe between source and network latency is that once a clock tree is built for the design, the network latency can be ignored. However the source latency remains even after the clock tree is built.
- The network latency is an estimate of the delay of the clock tree before clock tree synthesis. After clock tree synthesis, the total clock latency from the clock source to a clock in of a flip flop is the source latency plus actual delay of the clock tree from the clock definition point to the flip flop.

Clock Uncertainty:

- clock uncertainty is the difference between the arrivals of clocks at registers in one clock domain or between domains. it can be classified as static and dynamic clock uncertainties.
- Timing Uncertainty of clock period is set by the command set_clock_uncertainty at the synthesis stage to reserve some part of the clock period for uncertain factors (like skew, jitter, OCV, CROSS TALK, MARGIN or any other pessimism) which will occur in PNR stage. The uncertainty can be used to model various factors that can reduce the clock period.

- It can define for both setup and hold.
- Set_clock_uncertainty –setup 0.2 [get_clocks clk_name1]
- Set_clock_uncertainty –hold 0.05 [get_clocks clk_name1]
- Clock uncertainty for setup effectively reduces the available clock period by the specified amount as shown in fig. and the clock uncertainty for hold is used as an additional margin that needs to be satisfied.



- **Static clock uncertainty:** it does not vary or varies very slowly with time. Process variation induced clock uncertainty. An example of this is clock skew.

Sources of static clock uncertainty

- Intentional and unintentional mismatch in design
- On-chip variation (OCV)
- Load variation at every stage in clock distribution
- **Dynamic clock uncertainty**: it varies with time. Dynamic power supply induced delay variation and clock jitter is the example of this

Sources of dynamic clock uncertainty:

- Voltage droop and dynamic voltage variations
- Temperature variations
- Clock generator jitter

Jitter:

- Jitter is the short term variations of a signal with respect to its ideal position in time. It is the variation of the clock period from edge to edge.it can vary +/- jitter value. From cycle to cycle the period and duty cycle can change slightly due to the clock generation circuitry. This can be modeled by adding uncertainty regions around the rising and falling edge of the clock waveform.
- **Sources of jitter:**
- Internal circuitry of the PLL
- Thermal noise in crystal oscillators
- Transmitters and receivers of resonating devices

NOTE

- The first important point is that there are two phases in the design of when we are using a clock signal. In the first stage i.e. during RTL design, during synthesis and during placement the clock is ideal. The ideal clock has no distribution tree, it is directly connected at the same time to all flip flop clock pins
- The second phase comes when CTS inserts the clock buffer to build the clock tree into the design that carries the clock signal from the clock source pin to the all flip flops clock pins. After CTS is finished clock is called “propagated clock”.

- Clock latency term we are using when the clock is in ideal mode. It is the delay that exists from the clock source to the clock pin of the flip flop. This delay is specified by the user (not a real value or measured value).
- When the clock is in propagated mode the actual delay comes into the picture then this delay is called as insertion delay. Insertion delay is a real and measured delay path through a tree of buffers. Sometimes the clock latency is interpreted as a desired target value for insertion delay

Clock uncertainty

- in the ideal mode we assume the clock is arriving at all the flip flop at the same time but ideally, we did not get the clock at the same time, maybe the clock will arrive at different times at different clock pins of a flip flop so in ideal mode clock assume some uncertainty . for example a 1ns clock with 100 ps clock uncertainty means that next clock pulse will occur after 1ns±50ps (either + or -).

The question of why the clock does bit always arrive exactly after one clock?

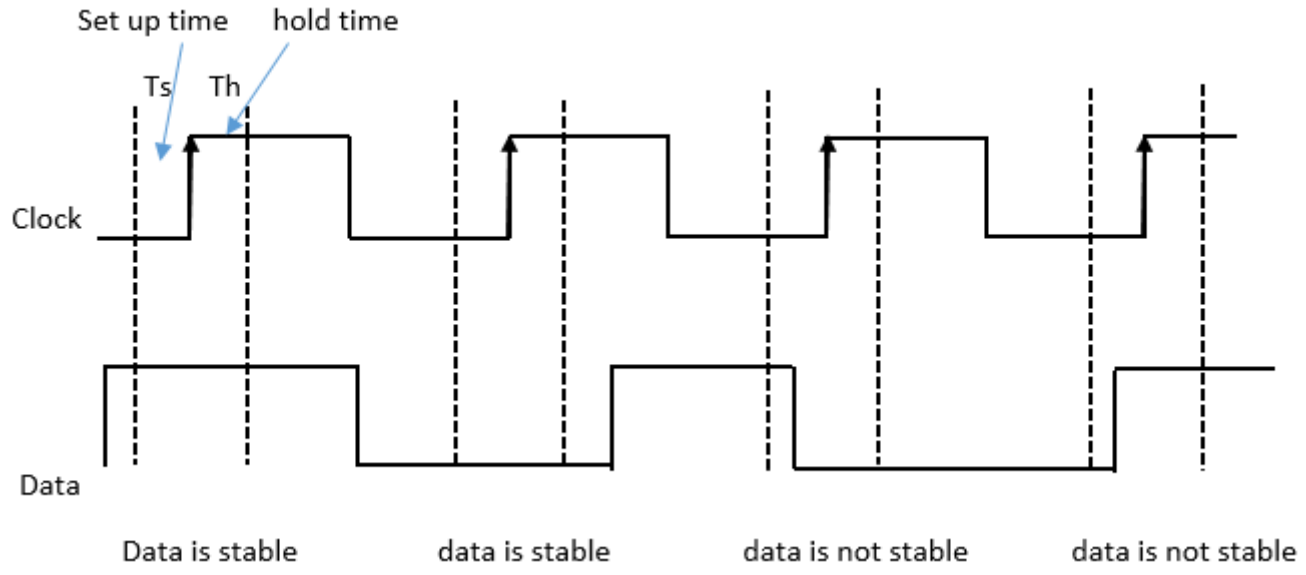
1.The insertion delay to the launching flip flop's clock pin is different than the insertion delay of capturing clock (like maybe capture clock is coming before then the launch clock or capture clock is coming after the launch clock that difference is called skew)

2. The clock period is not constant. Some clock cycles may are longer or shorter than others in a random fashion. This is called clock jitter.

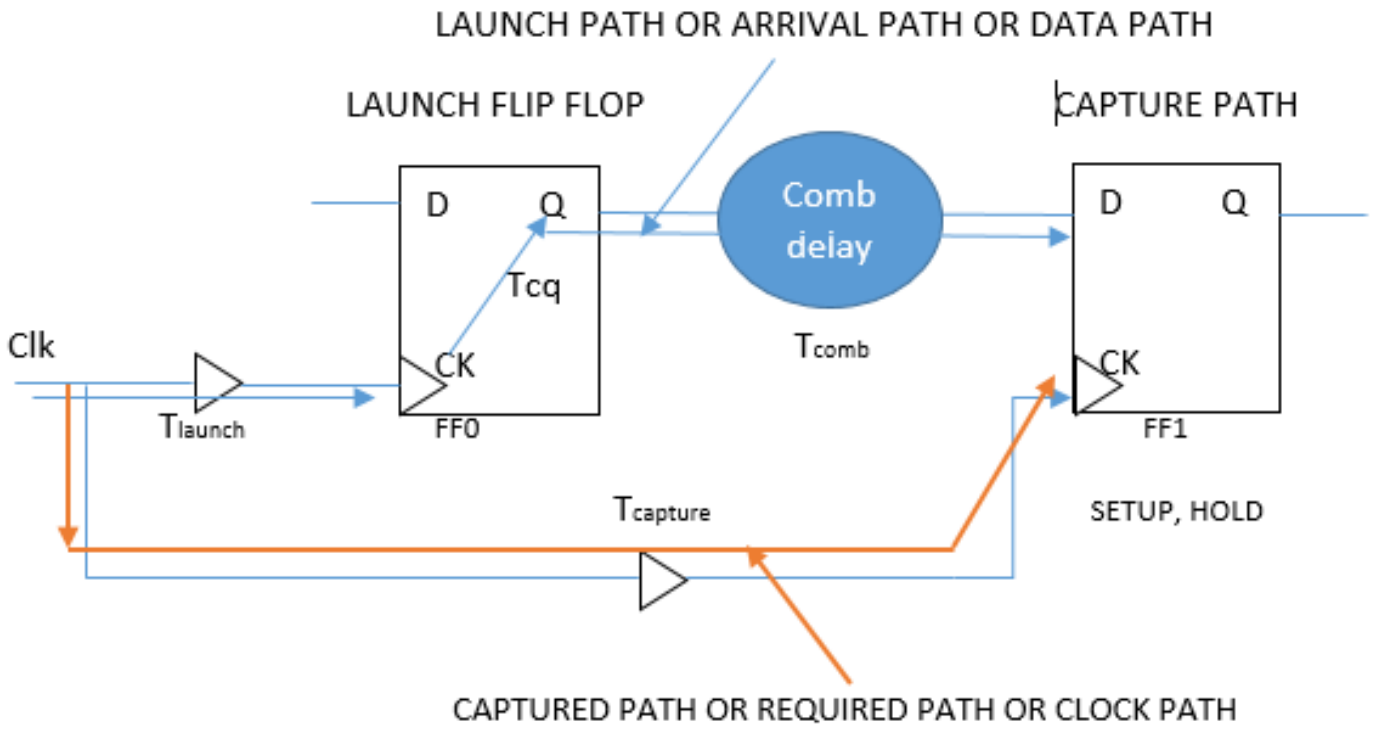
3. Even if the capture clock path and launch clock path are identical may be their path delays are different because different derate are applies on the path because the chip having different delay properties across the die due to process voltage and temperature variation i.e. called OCV (on-chip variation). This essentially increases the clock skew.

crosstalk and useful skew

- **Setup time:** The minimum time before the active edge of the clock, the input data should be stable i.e. data should not be changed at this time
- **Hold time:** The minimum time after the active edge of the clock, the input data should be stable i.e. data should not be changed at this time.



- **Capture edge:** the edge of the clock at which data is captured by a captured flip flop
- **Launch edge**: the edge of the clock at which data is launched by a launch flip flop



- **For setup check**
- Setup slack check = (required time) min – (arrival time) max
- Arrival time = Tlaunch + Tcq+ Tcomb
- Required time = Tclk+Tcapture-Tsu
- For setup time should not violate the required time should be greater than arrival time.
- **For hold check**
- Hold slack check = (arrival time) min – (required time) max
- Arrival time = Tlaunch + Tcq + Tcomb
- Required time = Tcapture + Thold
- For hold time should not violate the arrival time should be greater than the required time.

Crosstalk noise

- noise refers to undesired or unintentional effect between two or more signals that are going to affect the proper functionality of the chip. It is caused by capacitive coupling between neighboring signals on the die. In deep submicron technologies, noise plays an important role in terms of functionality or timing of device due to several reasons.
- Increasing the number of metal layers. For example, 28nm has 7 or 8 metal layers and in 7nm it's around 15 metal layers.
- Vertically dominant metal aspect ratio it means that in lower technology wire are thin and tall but in higher technology the wire is wide and thin, thus a greater the proportion of the sidewall capacitance which maps into wire to wire capacitance between neighboring wires.

- Higher routing density due to finer geometry means more metal layers are packed in close physical proximity.
- A large number of interacting devices and interconnect.
- Faster waveforms due to higher frequencies. Fast edge rates cause more current spikes as well as greater coupling impact on the neighboring cells.
- Lower supply voltage, because the supply voltage is reduced it leaves a small margin for noise.
- The switching activity on one net can affect on the coupled signal. The effected signal is called the victim and affecting signals termed as aggressors

- A **setup timing check** verifies the timing relationship between the clock and the data pin of a flip-flop so that the setup requirement is met.

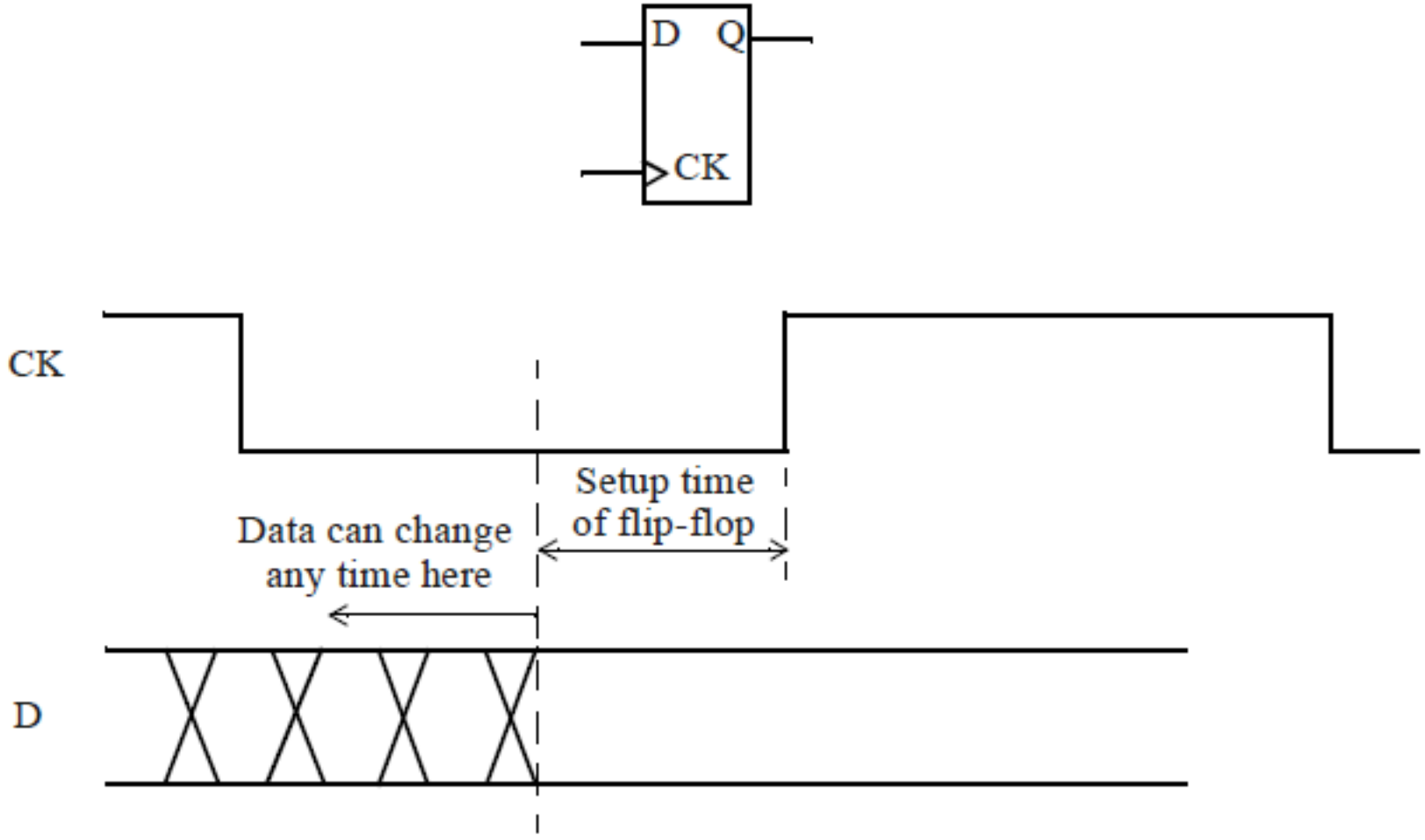
- the setup check ensures that the data is available at the input of the flip-flop before it is clocked in the flip-flop.

SETUP TIMING CHECK

- The data should be stable for a certain amount of time, namely the setup time of the flip-flop, before the active edge of the clock arrives at the flip-flop.

- This requirement ensures that the data is captured reliably into the flip-flop.

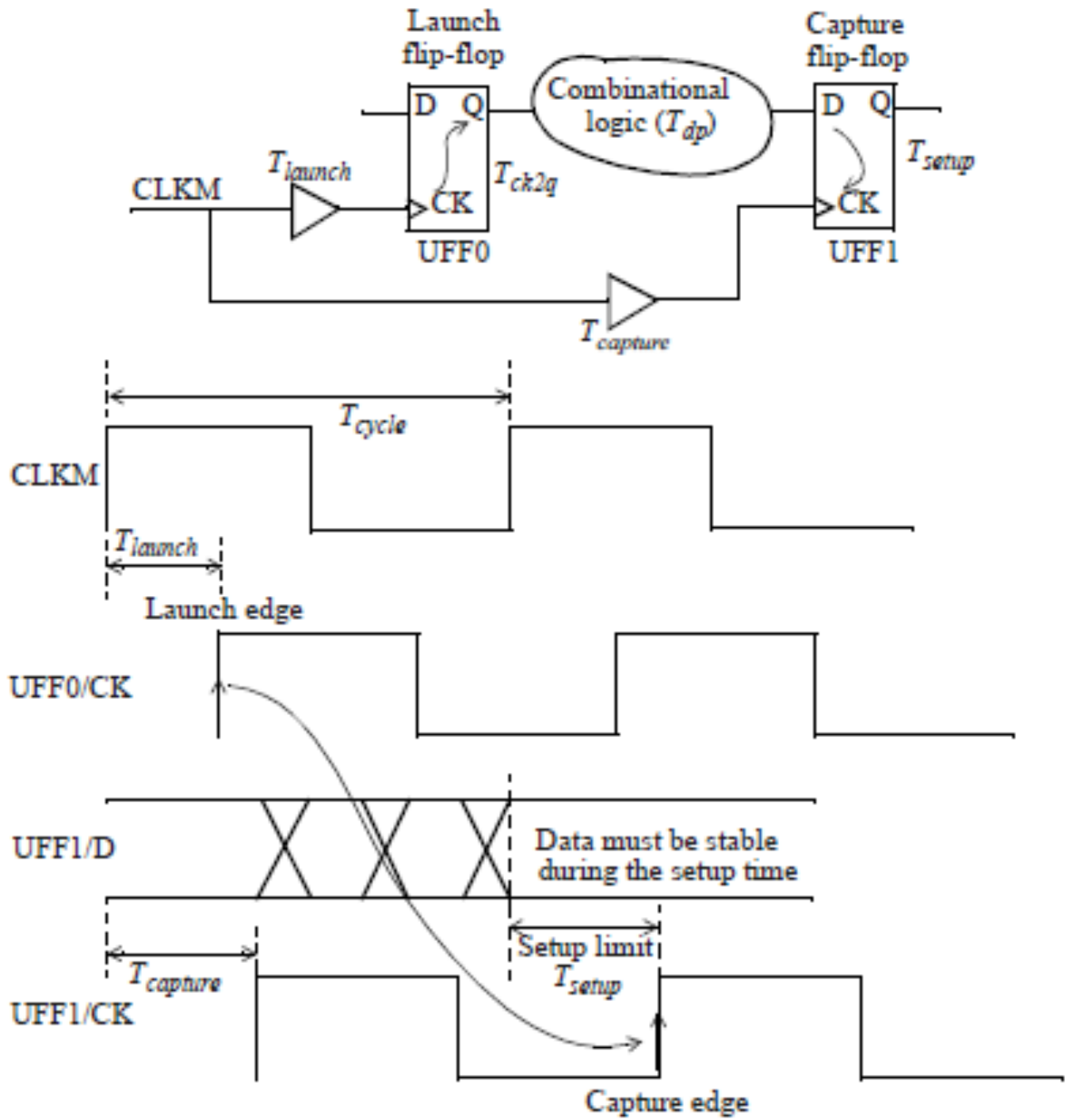
SETUP TIMING CHECK



ESSENCE OF SETUP CHECK

- The setup check is from the first active edge of the clock in the launch flip-flop to the closest following active edge of the capture flip-flop.

- The setup check ensures that the data launched from the previous clock cycle is ready to be captured after one cycle.


Launch
flip-flop
Capture
flip-flop
D
Q
Combinational
logic (T_{dp})
D
Q
T_{launch}
T_{ck2q}
T_{setup}
CLKM
CK
CK
UFF0
UFF1
$T_{capture}$
T_{cycle}
CLKM
T_{launch}
Launch edge
UFF0/CK
UFF1/D
Data must be stable
during the setup time
Setup limit
T_{setup}
$T_{capture}$
UFF1/CK
Capture edge

TRAVERSAL PATHS OF DATA AND CLOCK SIGNALS

- The data launched by this clock edge appears at time *Tlaunch* + *Tck2q* + *Tdp* at the *D* pin of the flip-flop *UFF1*.

- The second rising edge of the clock (setup is normally checked after one cycle) appears at time *Tcycle* + *Tcapture* at the clock pin of the capture flip-flop *UFF1*.

- The difference between these two times must be larger than the setup time of the flip-flop, so that the data can be reliably captured in the flip-flop.

TRAVERSAL PATHS OF DATA AND CLOCK SIGNALS

- From the above three statements we conclude that

$$Tlaunch + Tcktoq + Tdp < Tcapture + Tcycle - Tsetup$$

Means

$$Tcapture + Tcycle - (Tsetup + Tlaunch + Tcktoq + Tdp) > 0$$

WHERE SHOULD SETUP CHECKS BE EVALUATED?

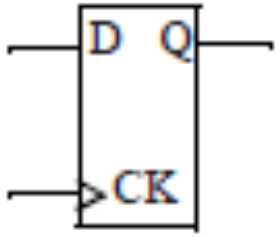
- Since the setup check poses a max constraint means upper bound on data path delay , the setup check *always* uses the longest or the max timing path. For the same reason, this check is normally verified at the slow corner where the delays are the largest.

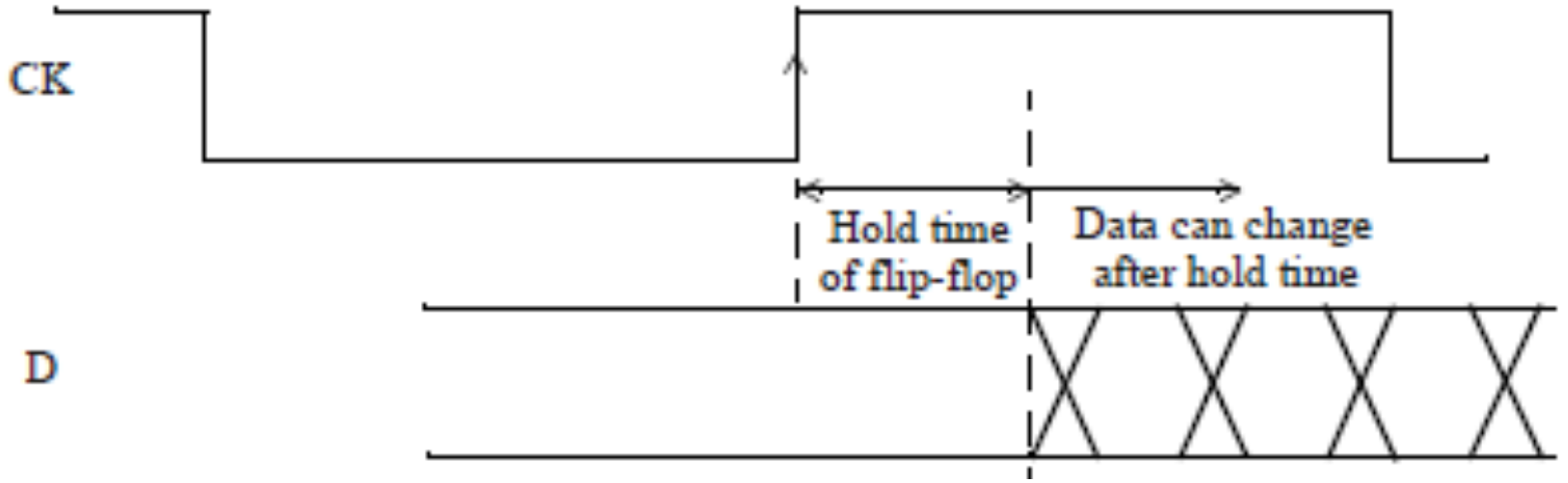
HOLD TIMING CHECK

- A **hold timing check** ensures that a flip-flop output value that is changing does not pass through to a capture flip-flop and overwrite its output before the flip-flop has had a chance to capture its original value.

- The hold specification of a flip-flop requires that the data being latched should be held stable for a specified amount of time after the active edge of the clock.

HOLD REQUIREMENT OF A FLIP FLOP

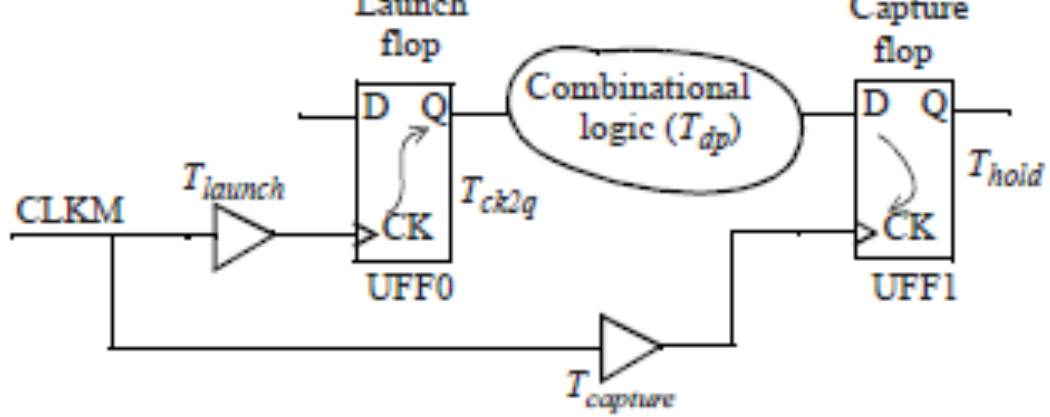


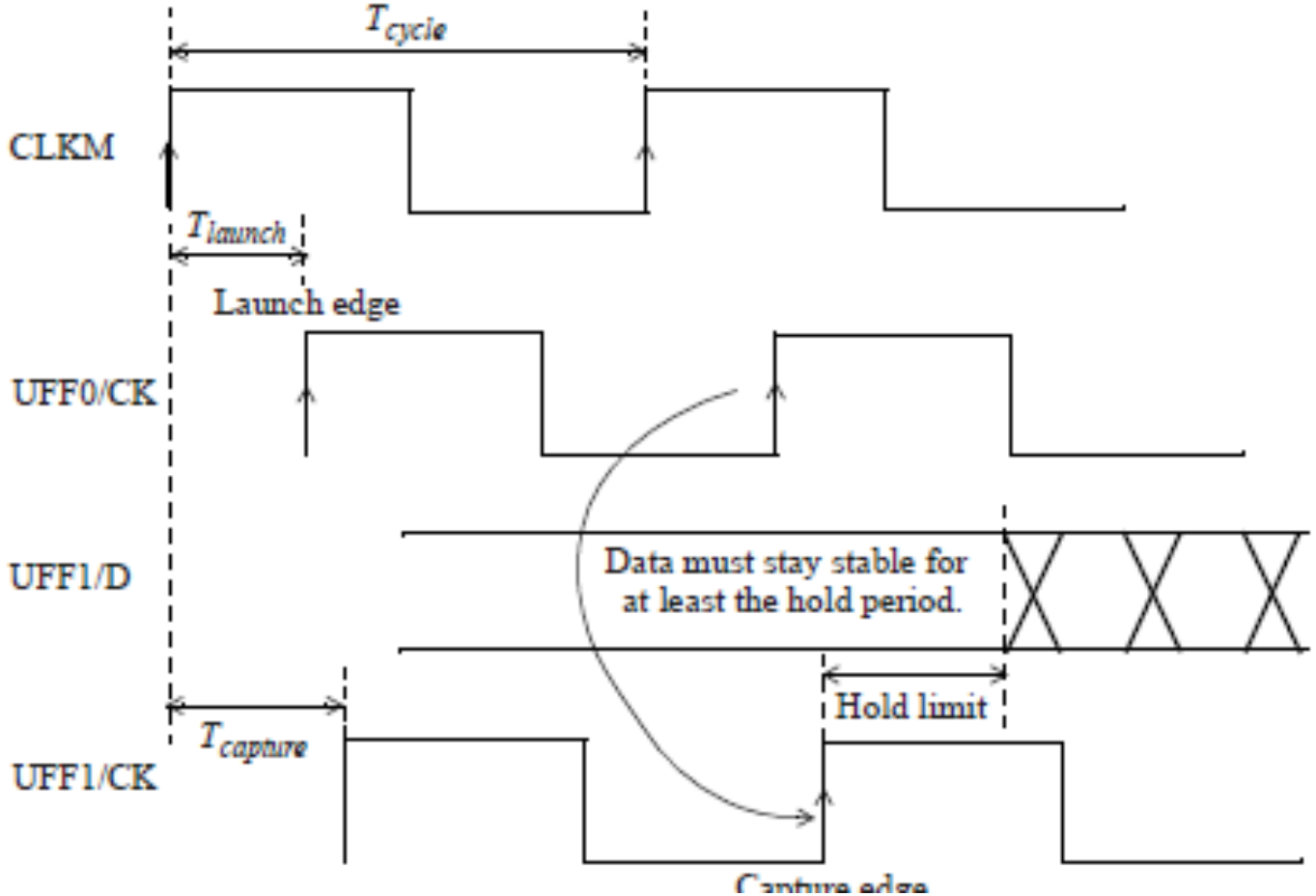


ESSENCE OF HOLD CHECK

- The hold check is from one active edge of the clock in the launch flip-flop to the same clock edge at the capture flip-flop.

- Thus, a hold check is independent of the clock period. The hold check is carried out on each active edge of the clock of the capture flip-flop.


Launch flop
Capture flop
D Q
Combinational logic (T_{dp})
D Q
T_{launch}
T_{ck2q}
T_{hold}
CLKM
CK
CK
UFF0
UFF1
$T_{capture}$


T_{cycle}
CLKM
T_{launch}
Launch edge
UFF0/CK
UFF1/D
Data must stay stable for at least the hold period.
Hold limit
$T_{capture}$
UFF1/CK
Capture edge

TRAVERSAL PATHS OF DATA AND CLOCK SIGNALS

- Consider the second rising edge of clock *CLKM*. The data launched by the rising edge of the clock takes *Tlaunch* + *Tcktoq* + *Tdp* time to get to the *D* pin of the capture flip-flop *UFF1*.

- The same edge of the clock takes *Tcapture* time to get to the clock pin of the capture flip-flop.

- The intention is for the data from the launch flip-flop to be captured by the capture flip-flop in the next clock cycle.

TRAVERSAL PATHS OF DATA AND CLOCK SIGNALS

- If the data is captured in the same clock cycle, the intended data in the capture flip-flop from the previous clock cycle is overwritten.

- The hold time check is to ensure that the intended data in the capture flipflop is not overwritten.

TRAVERSAL PATHS OF DATA AND CLOCK SIGNALS

- The hold time check verifies that the difference between these two times i.e data arrival time and clock arrival time at capture flip-flop must be larger than the hold time of the capture flip-flop, so that the previous data on the flip-flop is not overwritten and the data is reliably captured in the flip-flop.

$$Tlaunch + Tck2q + Tdp > Tcapture + Thold$$

Means

$$Tlaunch + Tck2q + Tdp-(Tcapture + Thold) > 0$$

Where should hold timing check be evaluated?

- The hold checks impose a lower bound or min constraint for paths to the data pin on the capture flip-flop; the fastest path to the *D* pin of the capture flip-flop needs to be determined.

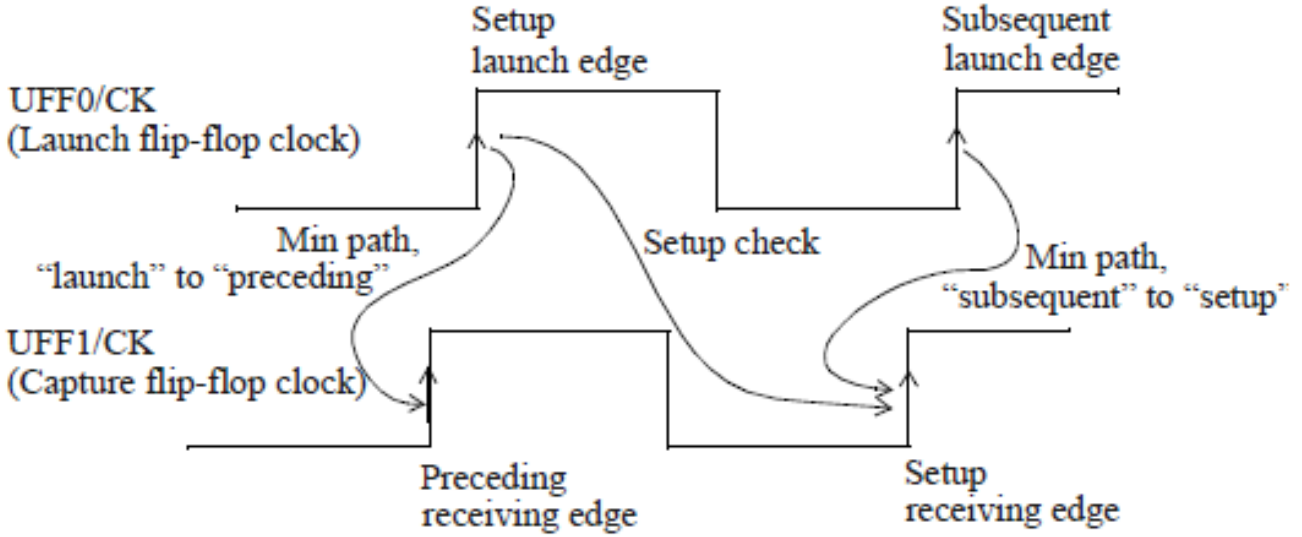
- This implies that the hold checks are *always* verified using the shortest paths. Thus, the hold checks are typically performed at the fast timing corner.

NOW LET US DEEP DIVE INTO CLOCK SKEW

- Even when there is only one clock in the design, the clock tree can result in the arrival times of the clocks at the launch and capture flip-flops to be substantially different. To ensure reliable data capture, the clock edge at the capture flip-flop must arrive before the data can change. A hold timing check ensures that

1. Data from the subsequent launch edge must not be captured by the setup receiving edge.

2.Data from the setup launch edge must not be captured by the preceding receiving edge.



Solution 1. The *subsequent launch edge* must not propagate data so fast that the *setup receiving edge* does not have time to capture its data reliably.

Solution2. the *setup launch edge* must not propagate data so fast that the *preceding receiving edge* does not get a chance to capture its data.

SKEW

- This phenomenon occurs in synchronous circuits. The Difference in arrival of clock at two consecutive pins of a sequential element.

Positive skew

- This phenomenon occurs when capture clock comes late than launch clock

NOW LET US DERIVE SETUP AND HOLD SLACKS FOR POSITIVE SKEW

- Setup slack=Required time-Arrival time
- Where required time is the time within which data should arrive at capture flop=T_{clk}-t_{setup}+t_{skew}
- Arrival time is the time which is taken by the data to actually arrive at the capture flop=T_{min}=T_{clq}+t_{comb}
- so setup slack=T_{clk}+t_{skew}-(t_{clq}+t_{comb}+t_{setup})
- **CONCLUSION:** setup slack is going to improve when there is a positive skew

- Now the required time becomes T-Tsu+Tskew.
- If there is a positive skew it means we are giving more time to data to arrive at D pin of capture FF.

Effect of positive skew on hold slack

- The arrival time of this $(n+1)^{th}$ data should at least be greater than the Thold time of capture flop FF2. Basically this current data (n) should be held for enough time for it to be captured reliably, that enough time is called hold time.

- nth data has to be stable at the capture clock for Tskew+ Thold time otherwise data n will be corrupted. So we can say +ve skew is bad for hold.

- Hold slack=Arrival time-Required time.
- Arrival time is the time which is taken by the data to actually arrive at the capture flop=$T_{min}=T_{clq}+t_{comb}$
- Where required time is the time within which data should arrive at capture flop=$T_{hold}+t_{skew}$
- So, hold slack =$T_{clq}+t_{comb}-T_{hold}-t_{skew}$

- **CONCLUSION:** positive skew is going to worsen hold slack

Negative skew

- if the capture clock comes early than the launch clock.

NOW LET US DERIVE SETUP AND HOLD SLACKS FOR NEGATIVE SKEW

- Setup slack=Required time-Arrival time

- Where required time is the time within which data should arrive at capture flop=T_{clk}-t_{setup}-t_{skew}

- Arrival time is the time which is taken by the data to actually arrive at the capture flop=T_{min}=T_{clq}+t_{comb}

- so setup slack=T_{clk}-t_{skew}-(t_{clq}+t_{comb}+t_{setup})
- **CONCLUSION** :setup slack is going to worsen.

EFFECT OF NEGATIVE SKEW ON HOLD SLACK

- Hold slack=Arrival time-Required time.
- Arrival time is the time which is taken by the data to actually arrive at the capture flop=$T_{min}=T_{clq}+t_{comb}$
- Where required time is the time within which data should arrive at capture flop=$T_{hold}-t_{skew}$
- So, hold slack =$T_{clq}+t_{comb}-T_{hold}+t_{skew}$

- **CONCLUSION:** negative skew is going to improve hold slack

THANK YOU